

Register Transfer & Microoperations

↳ Operations performed on data in registers in one clock pulse are called microoperations

$$R \leftarrow f(R, R)$$

where, f can be shift, load, increment, ...

↳ Can be used to describe the sequence of microoperations

↳ It is a:

1. Symbolic Language
2. A convenient tool for describing the internal organization of digital computers
3. Can also be used to facilitate the design process of digital systems

Organization of a digital System

↳ can be defined by specifying:

1. Set of registers and their functions
2. Microoperation Set
3. Control that is responsible for the sequence of these microoperations

↳ Viewing a computer at this level is called register transfer level because we are focusing on:

1. The System's Registers
2. The data transformations in them
3. The data transfers b/w them

Designation of Registers

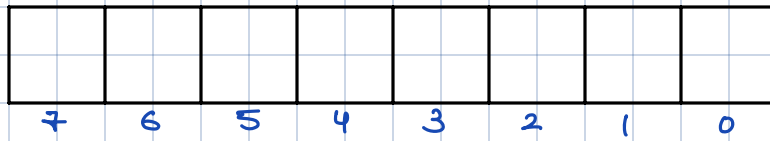
↳ By Capital letters sometimes followed by a number

↳ Can be viewed as:

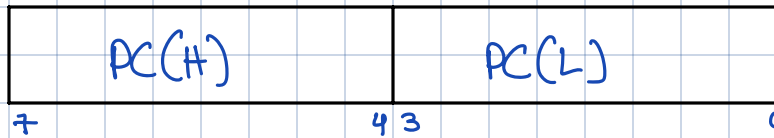
1. A single entity



2. Showing individual bits



3. As subfields



4. As a block

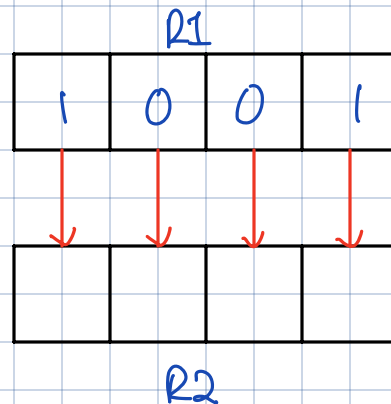


Register transfer

↳ Copying contents of one register to another

$$R2 \leftarrow R1$$

↳ Simultaneous transfer of all bits



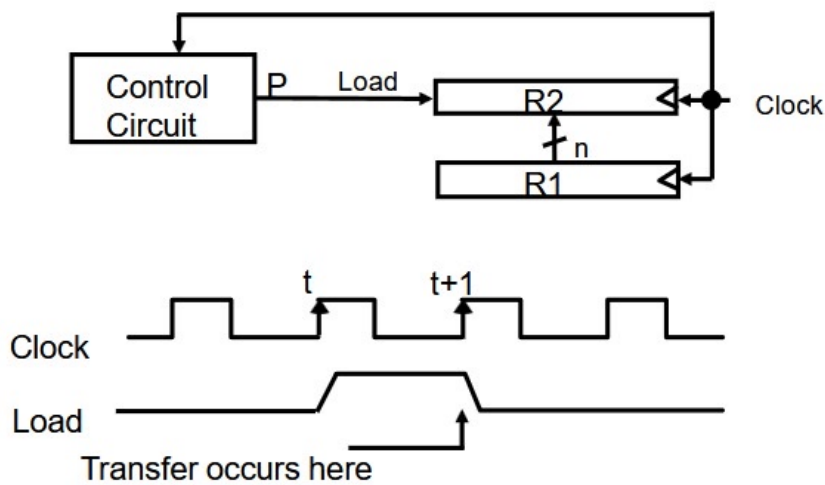
Control lines perform the action

Control functions:

↳ Actions occur only if a certain condition is true

$$P: R2 \leftarrow R1$$

↳ When $P=1$, transfer occurs



↳ More than one operation can also occur

P: $R3 \leftarrow R5, MAR \leftarrow IR$

Common Notations

Symbols	Description	Examples
Capital letters	Denotes a register	MAR, R2
Parentheses ()	Denotes a part of a register	$R2(0-7), R2(L)$
Arrow $\leftarrow$	Denotes transfer of information	$R2 \leftarrow R1$
Colon :	Denotes termination of control function	P:
Comma	Separates two micro-operations	$A \leftarrow B, B \leftarrow A$

Connecting Registers:

↳ If we have n registers, then we need $n(n-1)$ lines to connect all registers together.

↳ It is messy, makes it difficult to tell where the signal is coming from, unable to sequence signals

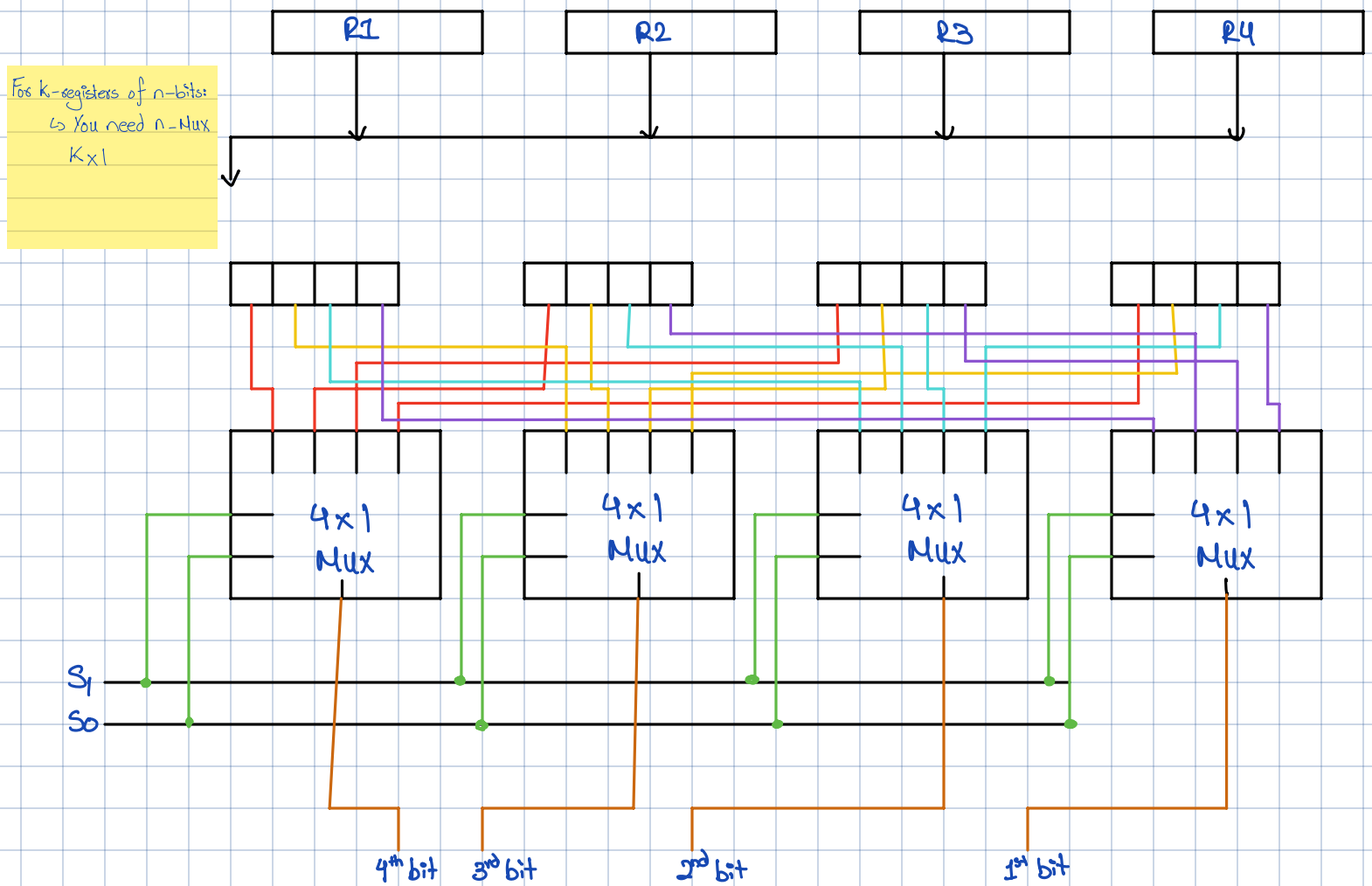
↳ So, we use bus

Bus

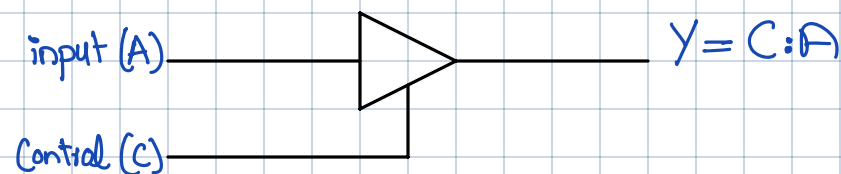
↳ It is path (of group of wires) over which information is

transferred from any several sources to any of several destinations.

Bus $\leftarrow R$



Three state buffer:



$$Y = 0$$

$$C = 1$$

$$Y = 1$$

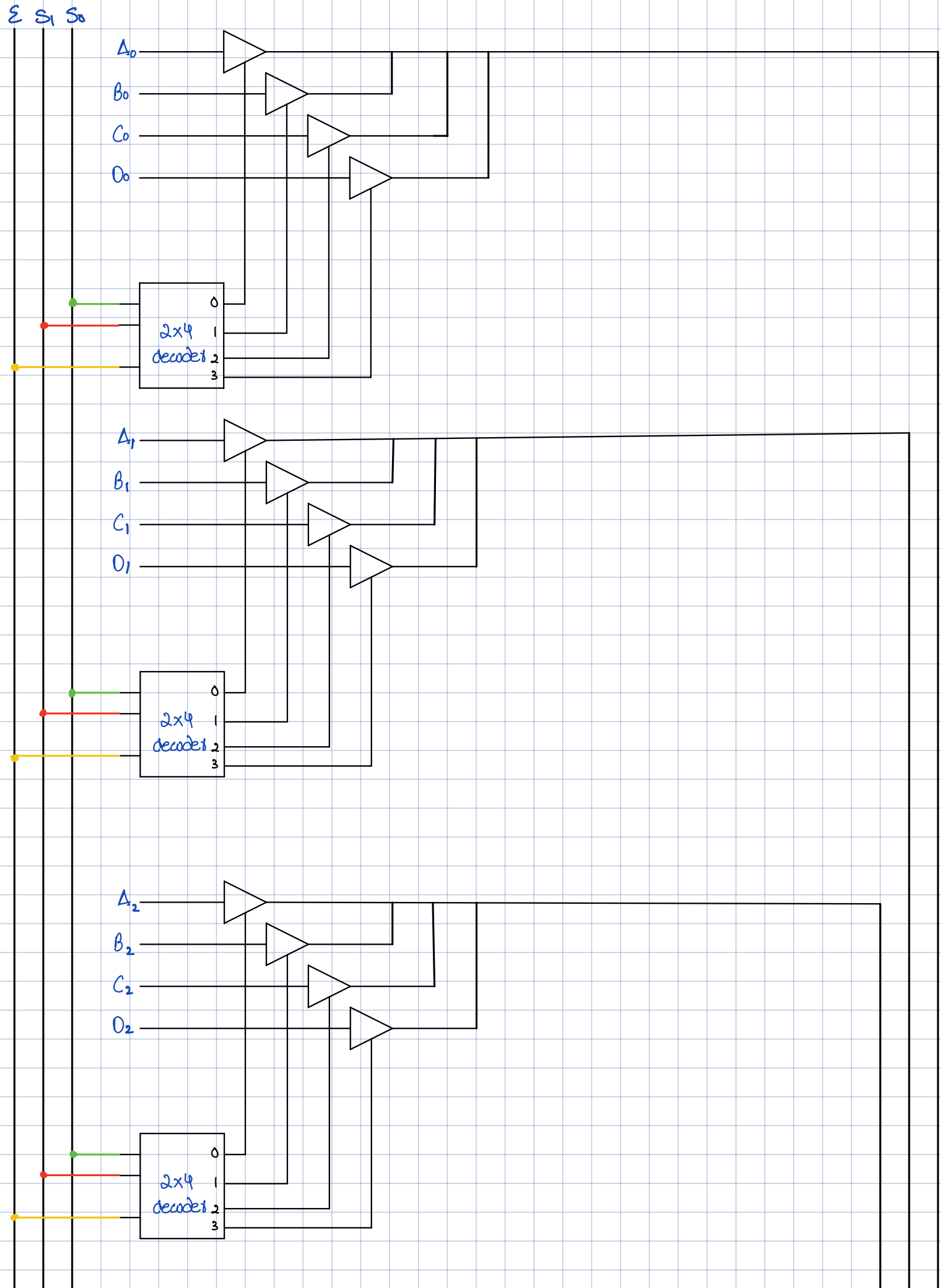
$$Y = \text{High impedance}, C = 0$$

Transfer from bus to register

$\hookrightarrow$ The above figure is complicated bound to increase in complexity as number of registers increase.

$\hookrightarrow$ So, we avoid Mux & opt for buffers and decoders.

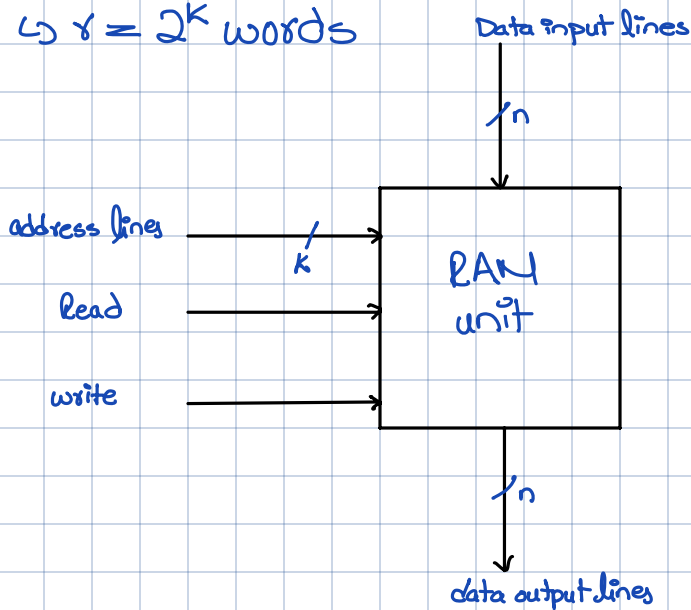
Register to bus



↳ Each of these registers x are indicated by an address ranging from 0 to $x-1$.

Let

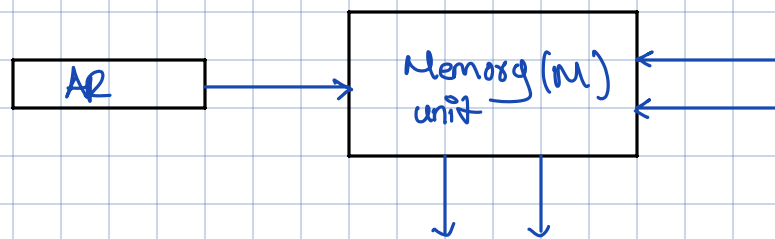
↳ $x = 2^k$ words



↳ Collectively viewed as a device M .

↳ Since there are multiple locations, we must specify which address in memory, we will be using. This is done by memory references

↳ Usually, accessed by putting the desired address in a special register. (MAR or AR) memory address register.



↳ To read a value from a location in memory and load it into a register,

$$R1 \leftarrow M[AR]$$

↳ Following occurs,

↳ Contents of AR are sent to address line

↳ Read is set to 1

↳ Contents at that location are put on memory's output data lines

↳ These are then sent to the register.

↳ To write a value from a register to a location in memory,

$$M[AR] \leftarrow R1$$

↳ Following occurs:

↳ The contents of AR are sent to address lines

↳ Write is set to 1.

↳ The values in register R1 are sent over data input lines of memory

↳ The values get loaded into the specified address in memory.

$A \leftarrow B$	Transfer content of reg. B into reg. A
$AR \leftarrow DR(LH)$	Transfer content of Lower portion of reg. DR into reg. AR
$A \leftarrow \text{constant}$	Transfer a binary constant into reg. A
$ABUS \leftarrow R1,$ $R2 \leftarrow ABUS$	Transfer content of R1 into bus A and, at the same time, transfer content of bus A into R2
AR	Address register
DR	Data register
$M[R]$	Memory word specified by reg. R
M	Equivalent to $M[AR]$
$DR \leftarrow M$	Memory <i>read</i> operation: transfers content of memory word specified by AR into DR
$M \leftarrow DR$	Memory <i>write</i> operation: transfers content of DR into memory word specified by AR

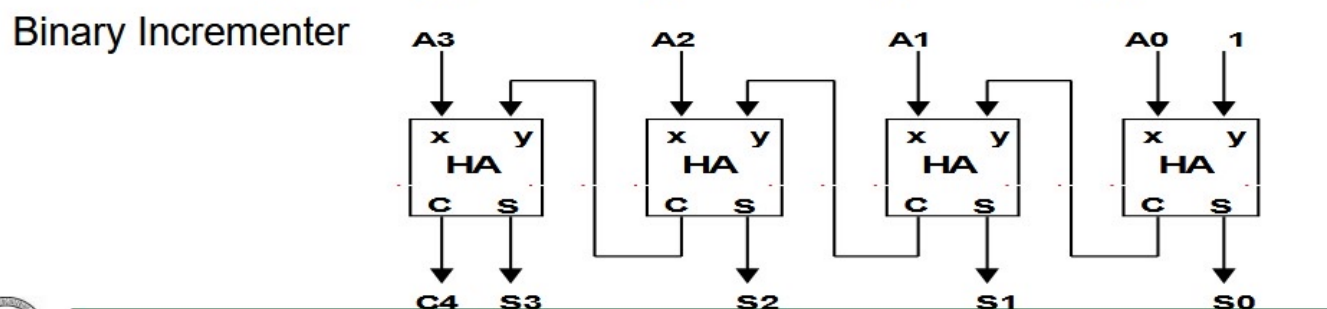
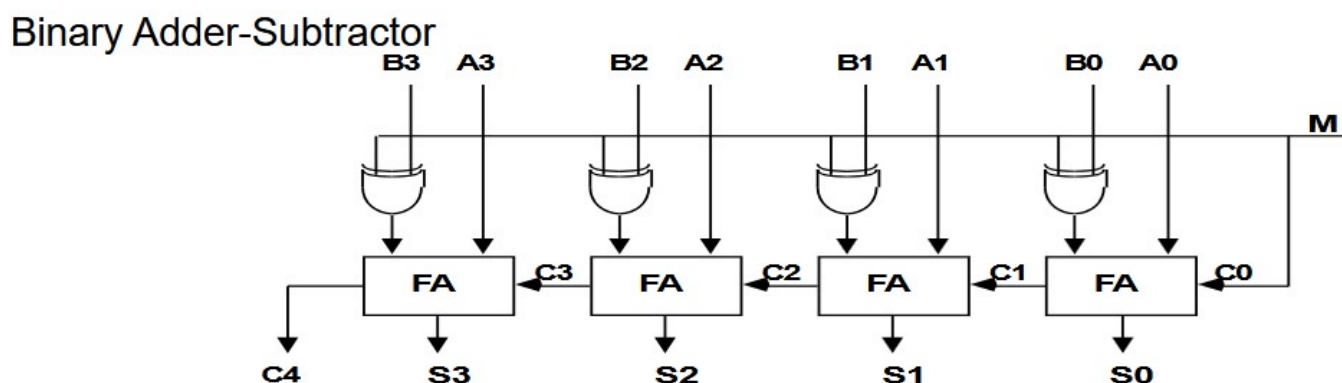
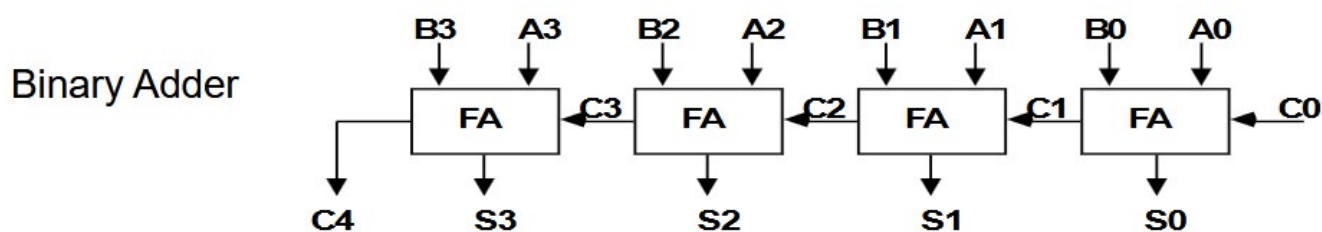
So far we have looked at register transfer microoperations.
Now, we will do the rest:

1. Arithmetic $\longrightarrow +, -$
2. Logic $\longrightarrow \wedge, \vee, \oplus, \sim$
3. Shift $\longrightarrow \text{Logical shift}, \div, *$

Summary of basic arithmetic operations

$R3 \leftarrow R1 + R2$	Contents of R1 plus R2 transferred to R3
$R3 \leftarrow R1 - R2$	Contents of R1 minus R2 transferred to R3
$R2 \leftarrow R2'$	Complement the contents of R2 (1's Complement)
$R2 \leftarrow R2' + 1$	2's complement the contents of R2 (Negate)
$R3 \leftarrow R1 + R2' + 1$	R1 plus 2's Complement of R2 (Subtraction)
$R1 \leftarrow R1 + 1$	Increment
$R1 \leftarrow R1 - 1$	Decrement

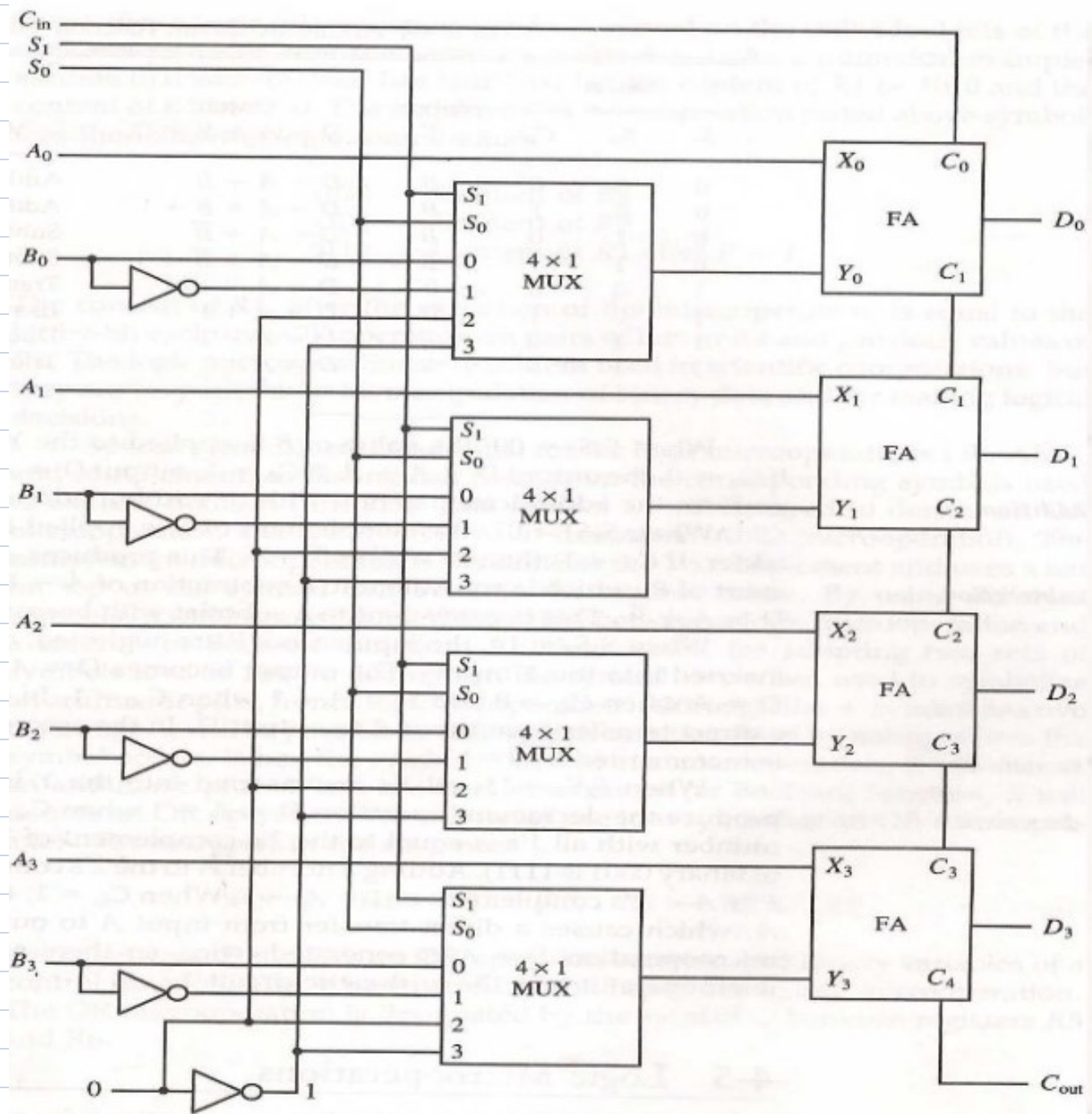
Arithmetic Circuits:



select		Cin	Input Y	Output $D = A + Y + C_{in}$	Microoperation
S ₁	S ₀				
0	0	0	B	A+B	Add
0	0	1	B	A+B+1	Add with carry
0	1	0	$\bar{B}$	A+ $\bar{B}$	Sub with borrow
0	1	1	$\bar{B}$	A+ $\bar{B}$ +1	Sub
1	0	0	0	A	Transfer
1	0	1	0	A+1	Increment
1	1	0	1	A-1	Decrement
1	1	1	1	A	Transfer

Note:

1111 is 2's complement of 0001.



Logic Micro-operations

Specify binary operations on bits of strings in a register

They are bitwise $\rightarrow A_i \wedge B_i$ i th bit

There are 16 functions defined over 2 binary inputs.

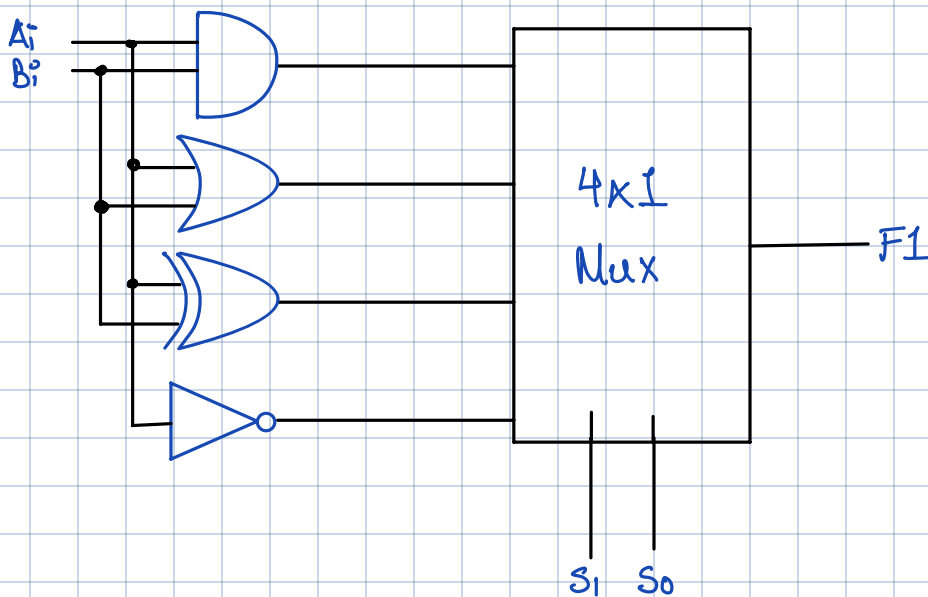
However, most systems implement only four:
AND, OR, XOR, NOT

Others are created from combinations of these.

↳ In general,
For n variables, there are 2^{2^n} functions

Boolean function	Microoperation	Name
$F_0 = 0$	$F \leftarrow 0$	Clear
$F_1 = xy$	$F \leftarrow A \wedge B$	AND
$F_2 = xy'$	$F \leftarrow A \wedge \overline{B}$	
$F_3 = x$	$F \leftarrow A$	Transfer A
$F_4 = x'y$	$F \leftarrow \overline{A} \wedge B$	
$F_5 = y$	$F \leftarrow B$	Transfer B
$F_6 = x \oplus y$	$F \leftarrow A \oplus B$	Exclusive-OR
$F_7 = x + y$	$F \leftarrow A \vee B$	OR
$F_8 = (x + y)'$	$F \leftarrow \overline{A \vee B}$	NOR
$F_9 = (x \oplus y)'$	$F \leftarrow \overline{A \oplus B}$	Exclusive-NOR
$F_{10} = y'$	$F \leftarrow \overline{B}$	Complement B
$F_{11} = x + y'$	$F \leftarrow A \vee \overline{B}$	
$F_{12} = x'$	$F \leftarrow \overline{A}$	Complement A
$F_{13} = x' + y$	$F \leftarrow \overline{A} \vee B$	
$F_{14} = (xy)'$	$F \leftarrow \overline{A \wedge B}$	NAND
$F_{15} = 1$	$F \leftarrow \text{all 1's}$	Set to all 1's

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1



S_0	S_1	F_1
0	0	$A \wedge B$
0	1	$A \vee B$
1	0	$A \oplus B$
1	1	$\overline{A}$

Application:

↳ Can be used to manipulate individual bits or portion of a word in a Register.

↳ Consider A, Processor Register, and B, Logic operand, is used to modify the contents of A.

1. Selective Set:

↳ Pattern in B is used to set bits in A.

$$A \leftarrow A \vee B$$

$$\begin{array}{r} 1100 \text{ } A_t \\ 1010 \text{ } B \\ \hline 1110 \text{ } A_{t+1} \end{array}$$

2. Selective Complement:

↳ Pattern in B is used to complement certain bits in A.

$$A \leftarrow A \oplus B$$

$$\begin{array}{r} 1100 \text{ } A_t \\ 1010 \text{ } B \\ \hline 0110 \text{ } A_{t+1} \end{array}$$

3. Selective Clear:

↳ Pattern in B is used to clear certain bits in A.

$$A \leftarrow A \wedge \bar{B}$$

$$\begin{array}{r} 1100 \text{ } A_t \\ 1010 \text{ } B \\ \hline 0101 \text{ } \bar{B} \\ 0100 \text{ } A_{t+1} \end{array}$$

Invert to make 1s where we want to clear bits and 0s where we want to keep bits unchanged.
targeted bits to 0, while others are same.

4. Mask Operation

↳ Extracts certain bits while clearing others

$$A \leftarrow A \wedge B$$

$$\begin{array}{r} 1100 \text{ } A_t \\ 1010 \text{ } B \\ \hline 1000 \text{ } A_{t+1} \end{array}$$

1s is where we want to keep bits unchanged
0s where we want to force to 0.

Why Can't We Use Masking for Selective Clear?

- At first, it might seem that masking ($R = R \& M$) can be used for selective clearing by setting 0s where we want to clear bits and 1s where we want to keep them. However, the issue comes down to flexibility and how it affects unintended bits.
- **Key Issue: Masking May Accidentally Clear Bits You Want to Keep**
 - If you use masking ($R = R \& M$) to clear specific bits, you must manually construct a mask that has:
 - 0s where you want to clear bits
 - 1s everywhere else (to preserve bits)
- This works, but it requires the mask to explicitly account for all bits you want to keep, which may not always be convenient.

- **Example Where Masking Fails**

- Suppose we have an 8-bit register and we want to clear only bit 3 while leaving everything else unchanged.
- **Using Masking ($R = R \& M$)**

- The mask (M) must be: 11110111
- Bit 3 is 0 (to clear it).
- All other bits are 1 (to preserve them).
- Register (R): 10101111
- Operation ($R = R \& M$):

```

  1 0 1 0 1 1 1 1
  1 1 1 1 0 1 1 1
  -----
  1 0 1 0 0 1 1 1

```

- This works but requires explicitly setting 1s in all other positions.

- **Using Selective Clear ($R = R \& \sim M$)**

- Instead of manually setting all 1s, we only specify which bits to clear using M, and let $\sim M$ handle the rest.
- Mask (M): 00001000 (1 only at the bit we want to clear)
- Complement ($\sim M$): 11110111 (Automatically preserves other bits)
- Operation ($R = R \& \sim M$):

```

  1 0 1 0 1 1 1 1
  0 0 0 0 1 0 0 0
  -----
  1 0 1 0 0 1 1 1

```

- **Same result**, but we only needed to specify which bits to clear instead of worrying about setting 1s elsewhere.

5. Compare:

↳ Compares A & B and produces 0 if they are equal

$$A \leftarrow A \oplus B$$

$$\begin{array}{r} 1100 A_t \\ 1010 B \\ \hline 0110 A_{t+1} \end{array}$$

6. Insert:

↳ Insert specific pattern into a register, leaving others unchanged

$$A \leftarrow (A \wedge M) \vee B$$

$$\begin{array}{r} 1101 \quad 1000 \quad 1011 \quad 0001 A_t \\ 1111 \quad 1111 \quad 1111 \quad 0000 M \\ \hline 1111 \quad 1000 \quad 1011 \quad 0000 \\ 0000 \quad 0000 \quad 0000 \quad 1010 B \\ \hline 1111 \quad 1000 \quad 1011 \quad 1010 A_{t+1} \end{array}$$

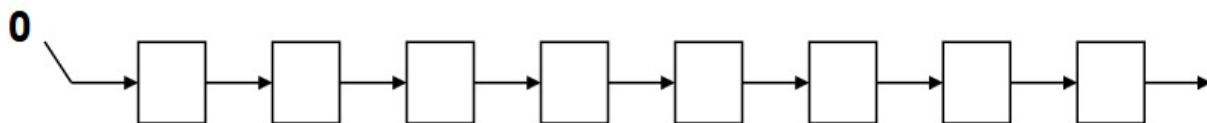
Shift Micro-operations

1. Logical shift:

↳ Serial input is set to 0

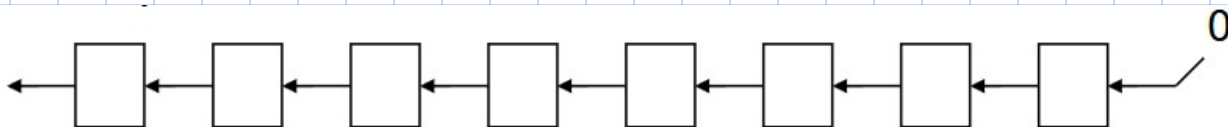
a. Right shift → Dividing by 2

$$R2 \leftarrow shr R2$$



b. Left Shift → Multiplying by 2

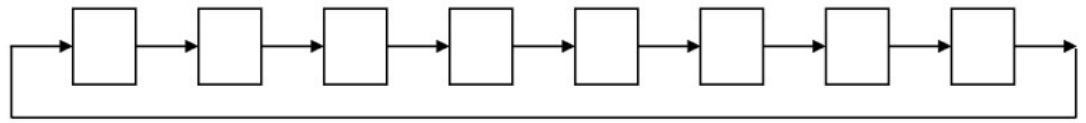
$$R2 \leftarrow shl R2$$



2. Circular shift:

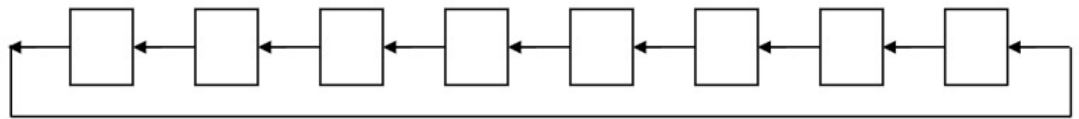
a. Right Circular shift

↳ $R2 \leftarrow \text{cir } R2$



b. Left circular shift

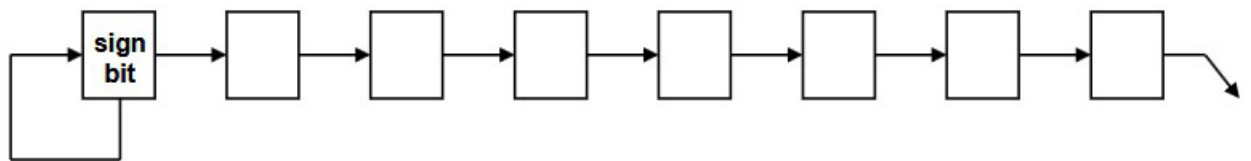
↳ $R2 \leftarrow \text{cil } R2$



3. Arithmetic shift:

↳ Meant for signed binary numbers

a. Right arithmetic shift:

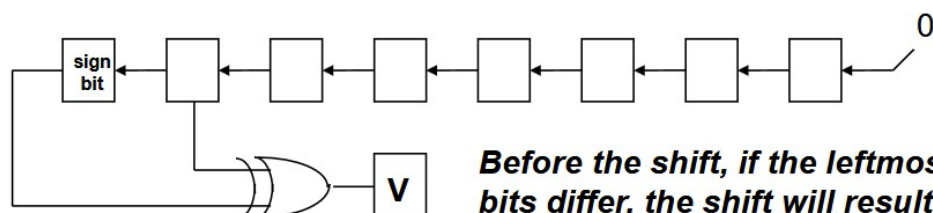


b. Left arithmetic shift:

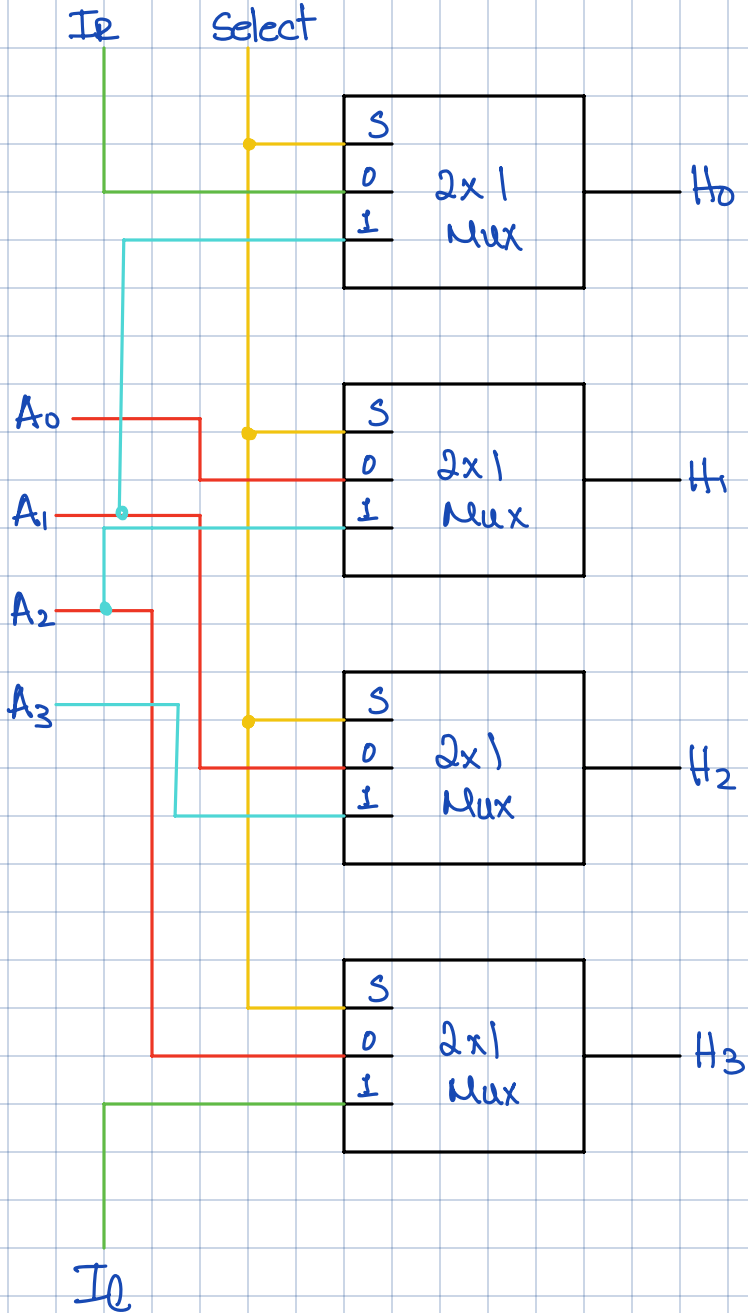
↳ Due to nature of LAS, sign reversal can occur which indicates integer overflow.

↳ For example,

-100 in binary when left shifted instead of doubling
-200 becomes 50.



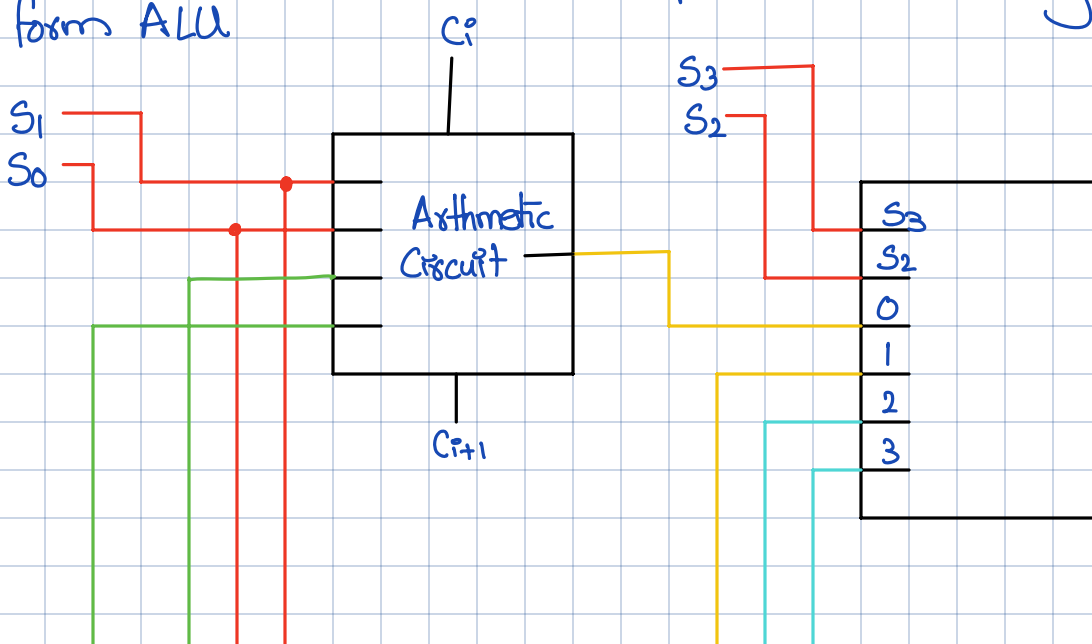
Before the shift, if the leftmost two bits differ, the shift will result in an overflow

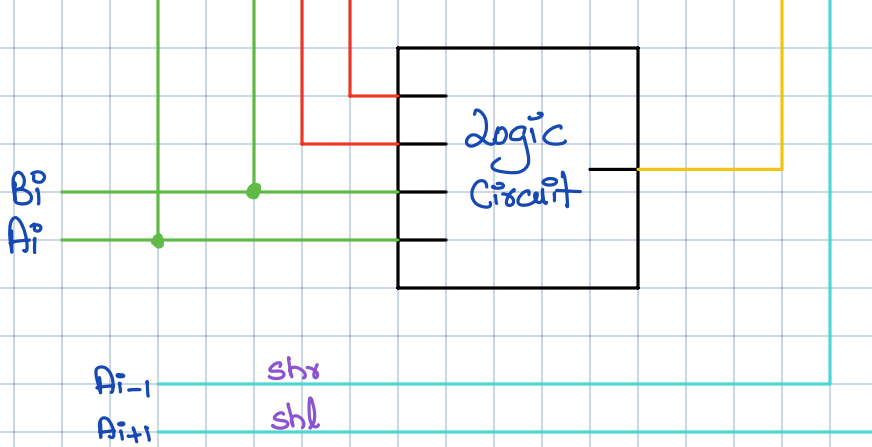


S	H_0	H_1	H_2	H_3
0	I_2	A_0	A_1	A_2
1	A_1	A_2	A_3	I_0

Arithmetic Logic Shift Unit

↳ Combines Arithmetic micro-operations with logical micro-operations to form ALU



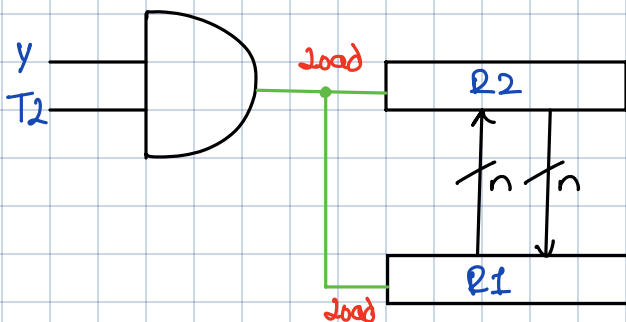


S_3	S_2	S_1	S_0	C_{in}	F	Operation
0	0	0	0	0	A	Transfer A
0	0	0	0	1	$A+1$	Increment A
0	0	0	1	0	$A+B$	Add
0	0	0	1	1	$A+B+1$	Add with carry
0	0	1	0	0	$A+\bar{B}$	Sub with borrow
0	0	1	0	1	$A+\bar{B}+1$	Sub
0	0	1	1	0	$A-1$	Decrement A
0	0	1	1	1	A	Transfer A
0	1	0	0	X	$A \wedge B$	AND
0	1	0	1	X	$A \vee B$	OR
0	1	1	0	X	$A \oplus B$	XOR
0	1	1	1	X	$\bar{A}$	Complement A
1	0	X	X	X	shr A	Shift right A
1	1	X	X	X	shl A	Shift left A

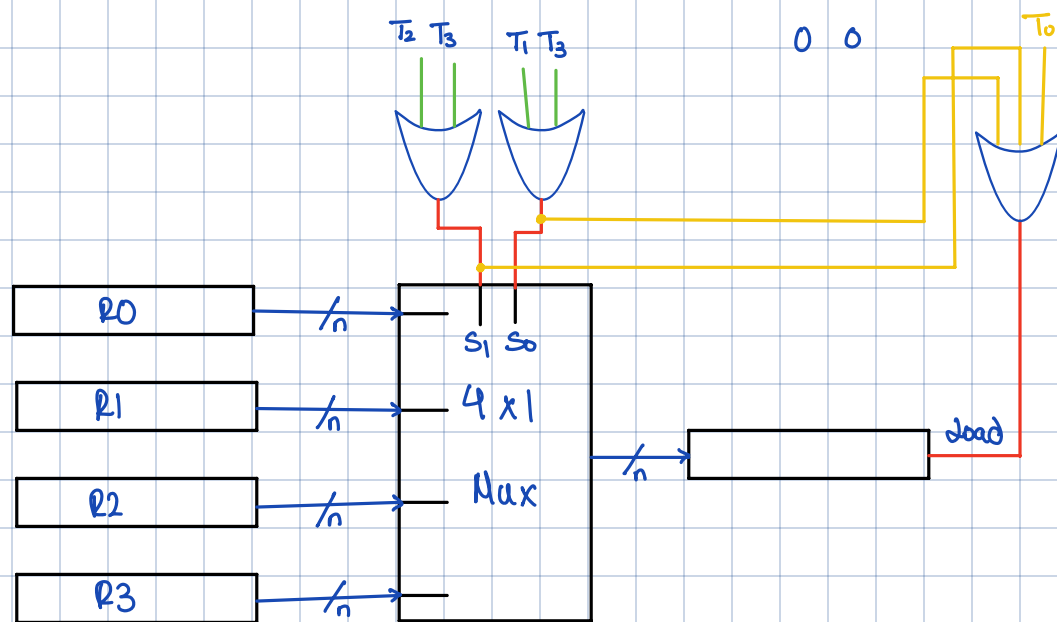
Book Qs:

Chapter 04:

Q1. $y_{T2}: R2 \leftarrow R1, R1 \leftarrow R2$



Q2.



T ₀	T ₁	T ₂	T ₃	S ₁	S ₀	L
0	0	0	0	X	X	0
1	0	0	0	0	0	1
0	1	0	0	0	1	1
0	0	1	0	1	0	1
0	0	0	1	1	1	1

Q3.

P: $R1 \leftarrow R2$

PQ: $R1 \leftarrow R3$

Q6.

16 register 32 bit

(b) 16×1

(a) $2^4 = 4$ selection bits

(c) 32

Q7.

(a) $R2 \leftarrow M[AR]$

↳ Value at Address AR transferred to R2

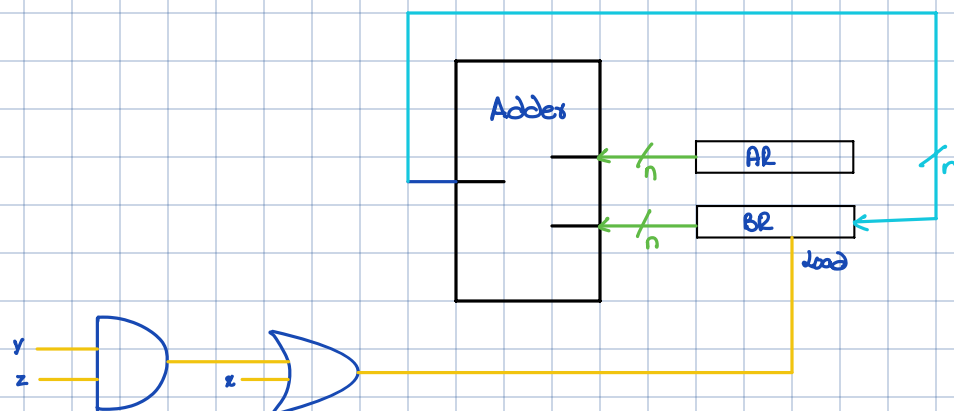
(b) $M[AR] \leftarrow R3$

↳ Contents of R3 transferred to memory at address in AR.

(c) $R5 \leftarrow M[R5]$

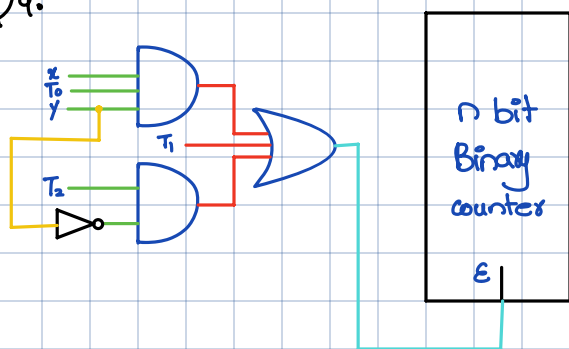
↳ Contents of M at R5

Q8.

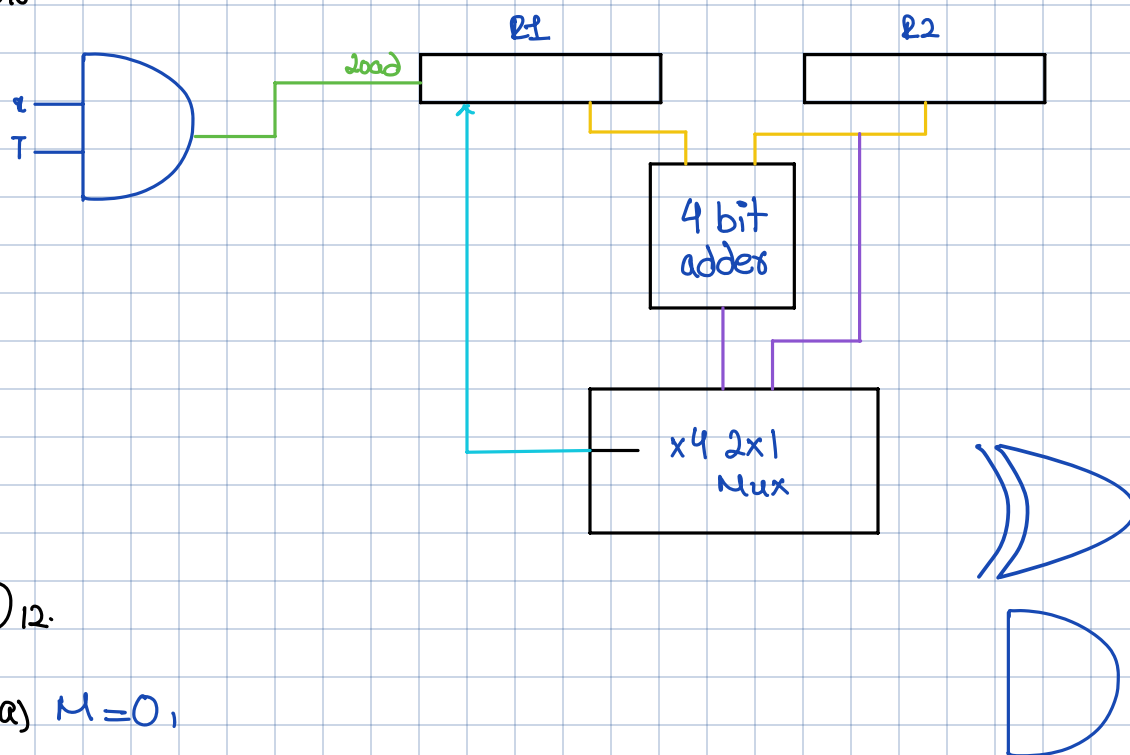


transferred to RS.

Q9.



Q10.



Q12.

(a) $M=0$,

0 1 1 0 B

$\oplus$

0 0 0 0 M

0 1 1 0 B

0 1 1 1

1 1 0 1

$S_3 S_2 S_1 S_0$

Q20.

1 1 0 0 1 1 1 0 $\rightarrow$ RS

0 0 1 1 1 0 0 0 $\rightarrow$ LS

Q₂₃.

(a) $xT : AR \leftarrow \overline{AR}, AR \leftarrow 0$

↳ Simultaneous access of same register.